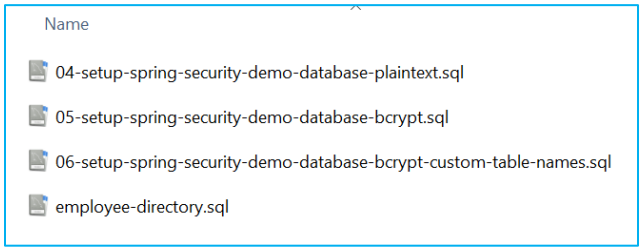
15. Spring MVC Security - JDBC Authentication - Plain Text – Coding

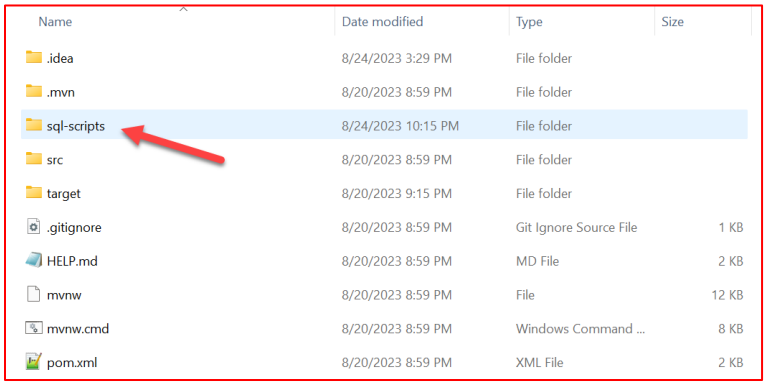
JDBC Authentication with Spring Security

First, please download the starter file attached to this lesson. Pause reading this document, download the file, and then continue when you're ready.

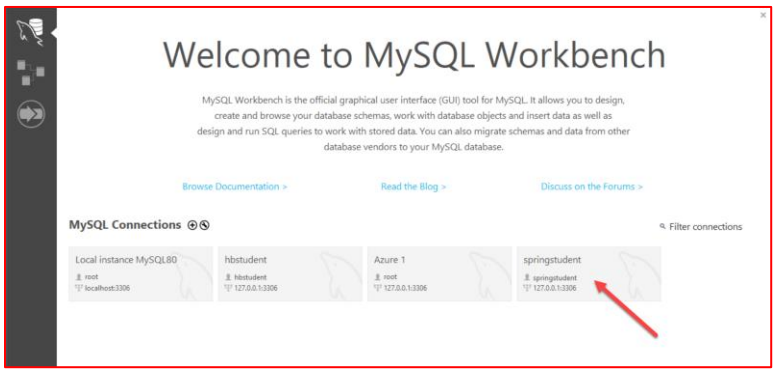
Now that you've downloaded the file, navigate to your download's directory. You'll find a zipped folder which contains another folder named `"SQL scripts"`. These scripts will be essential for upcoming sessions.



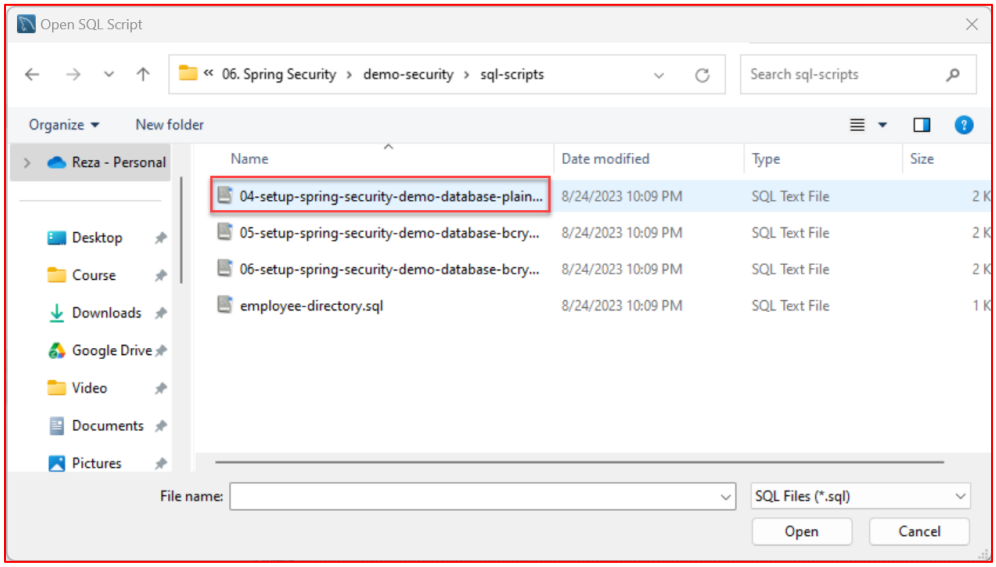
We'll head into our project folder We'll then copy the `'SQL scripts'` directory and paste it inside our project directory.



With that set up, let's launch MySQL Workbench and log in.



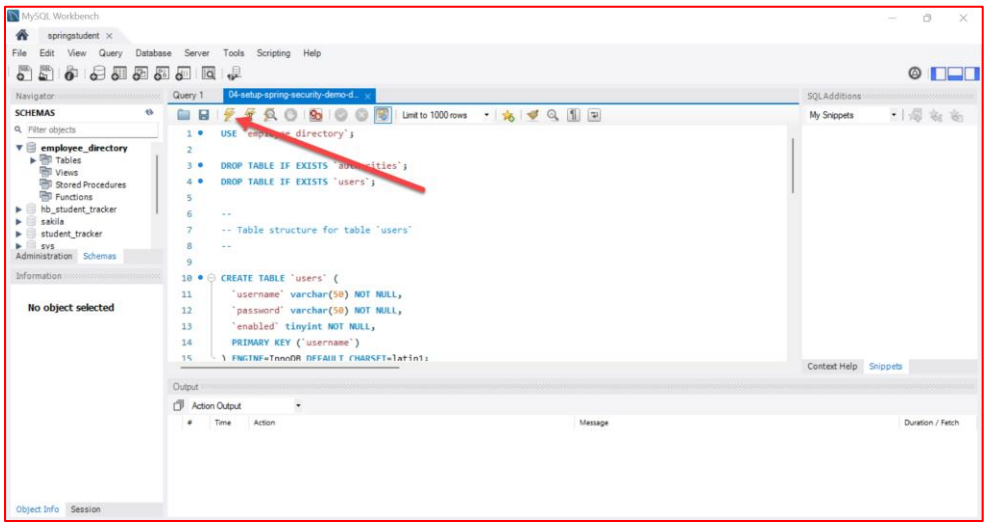
Within MySQL Workbench, navigate to the aforementioned directory (choose Open SQL Script) and select `'04-setup-spring-security-demo-database-plaintext.sql'`. This script will help us establish our tables and populate them.



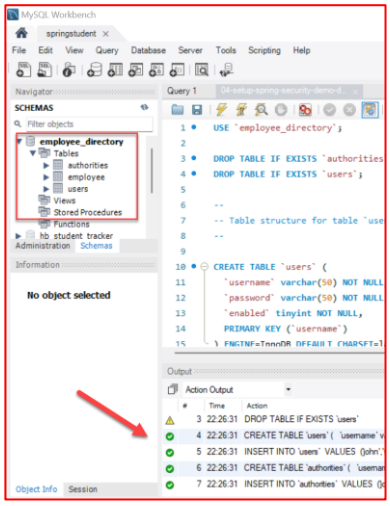
To summarize, we'll:

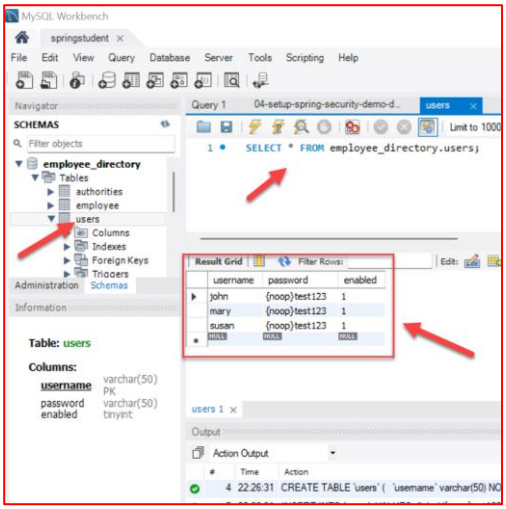
1. Drop existing tables.
2. Create a table for users with columns for `username`, `password`, and `enabled` status.
3. Populate the table with sample data for users: `John`, `Mary`, and `Susan`.
4. Remember: for the passwords, the prefix `'noop'` in curly braces indicates that the passwords are stored as plain text. We will delve into encryption in subsequent sections.
5. Create an `authorities` table and insert role data. Spring Security auto-prefixes roles with `'role_'`.

After setting up the database, it's time to verify our work.



Refresh and you should observe the new `'users'` and `'authorities'` tables.

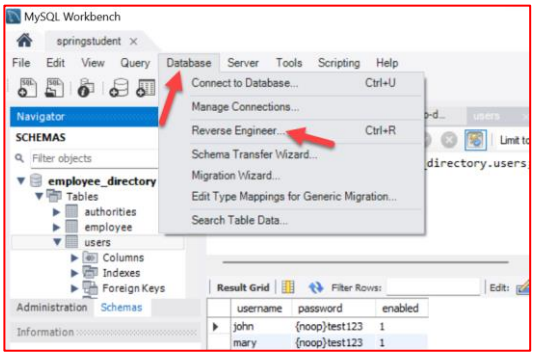




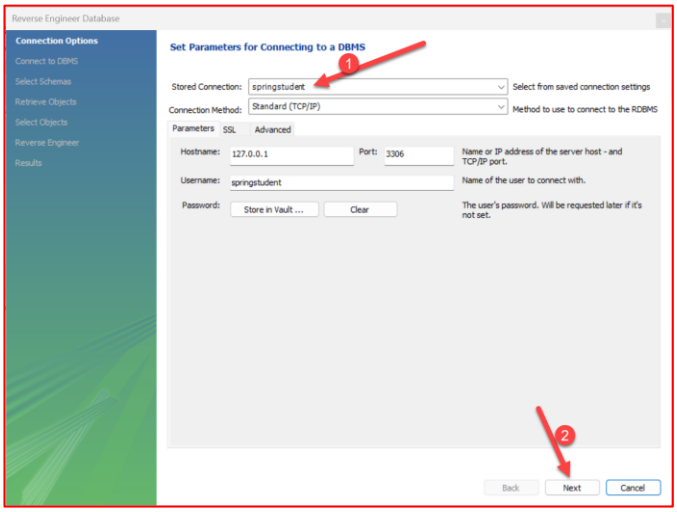
Feel free to query them and observe the assigned roles for each user.

For better visualization, we'll showcase how to create a database diagram. Using the `'reverse engineer'` feature, we can observe the relationships between the tables.

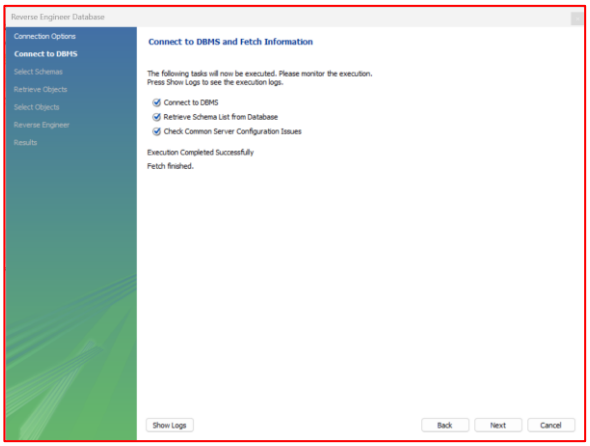
Step1



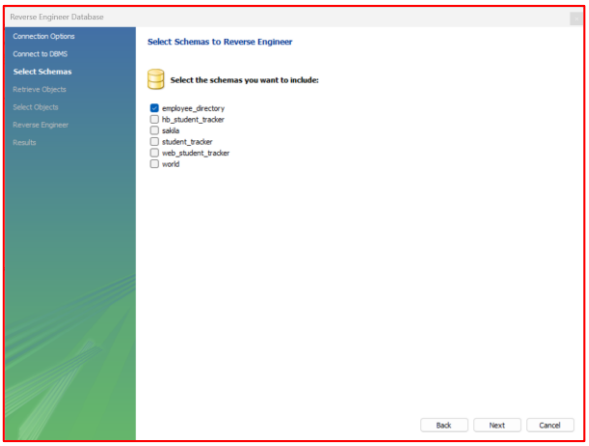
Step 2



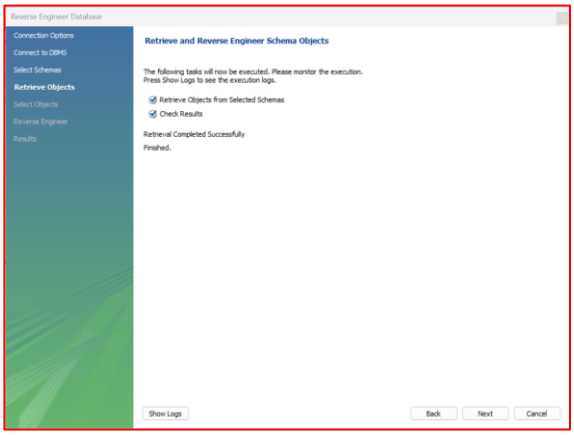
Step 3



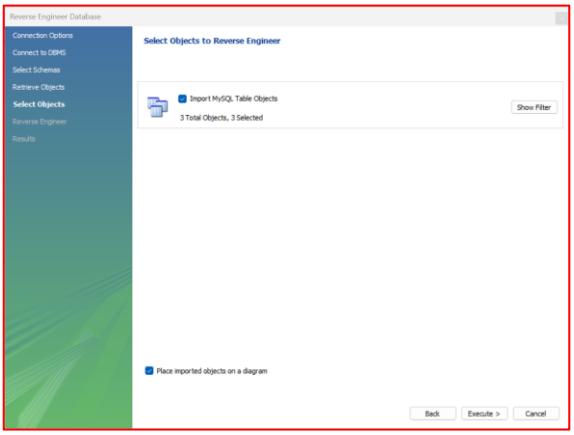
Step 4



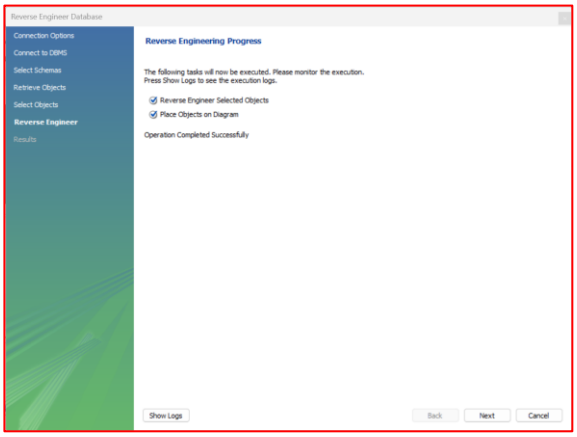
Step 5



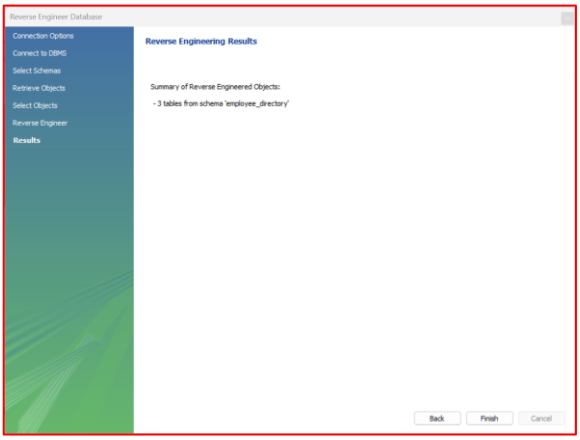
Step 6



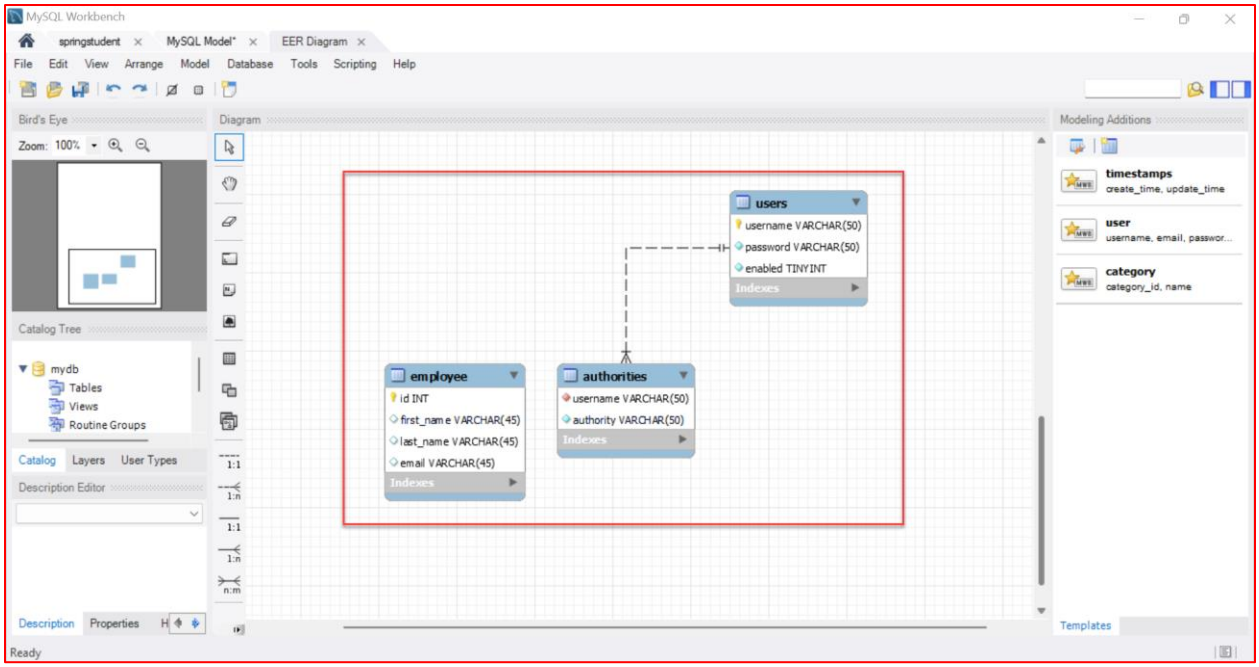
Step 7



Step 8



Here, we notice the connection between the `'user'` table and the `'authorities'` table, demonstrating that a user can be associated with multiple roles.



With our database ready, in the next session, we'll dive into coding.

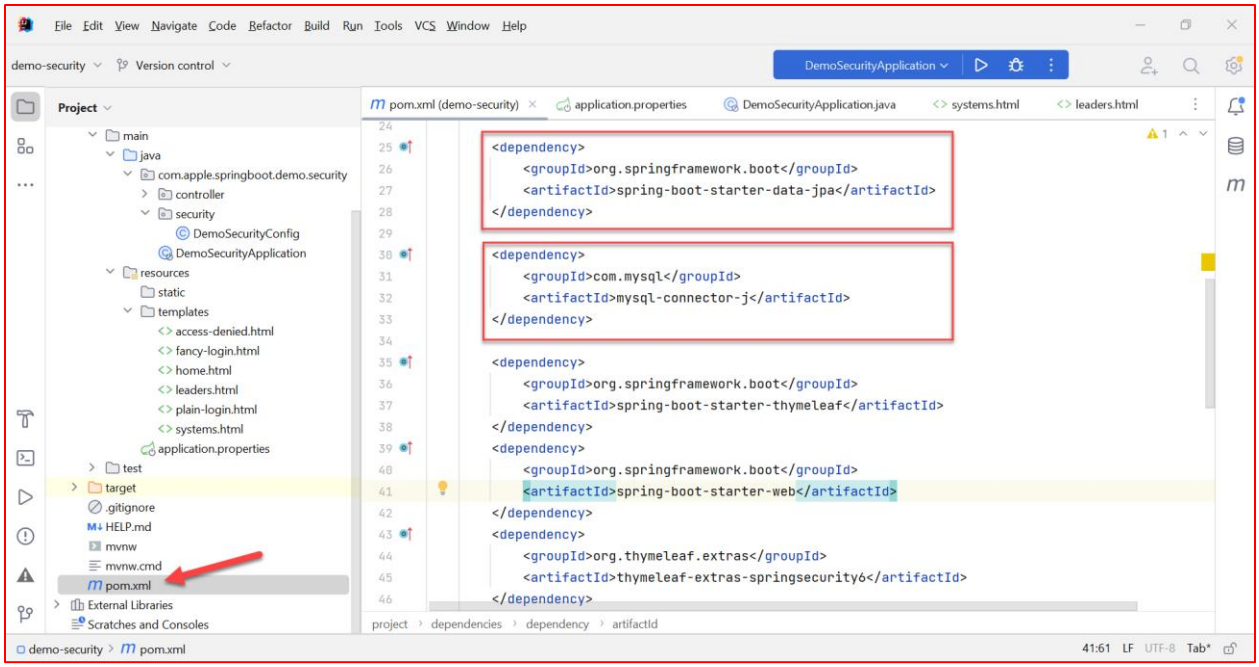
Begin by adding Maven dependencies to your `'pom.xml'` file. Introduce the `'spring-boot-starter-data-jpa'` for database access,

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

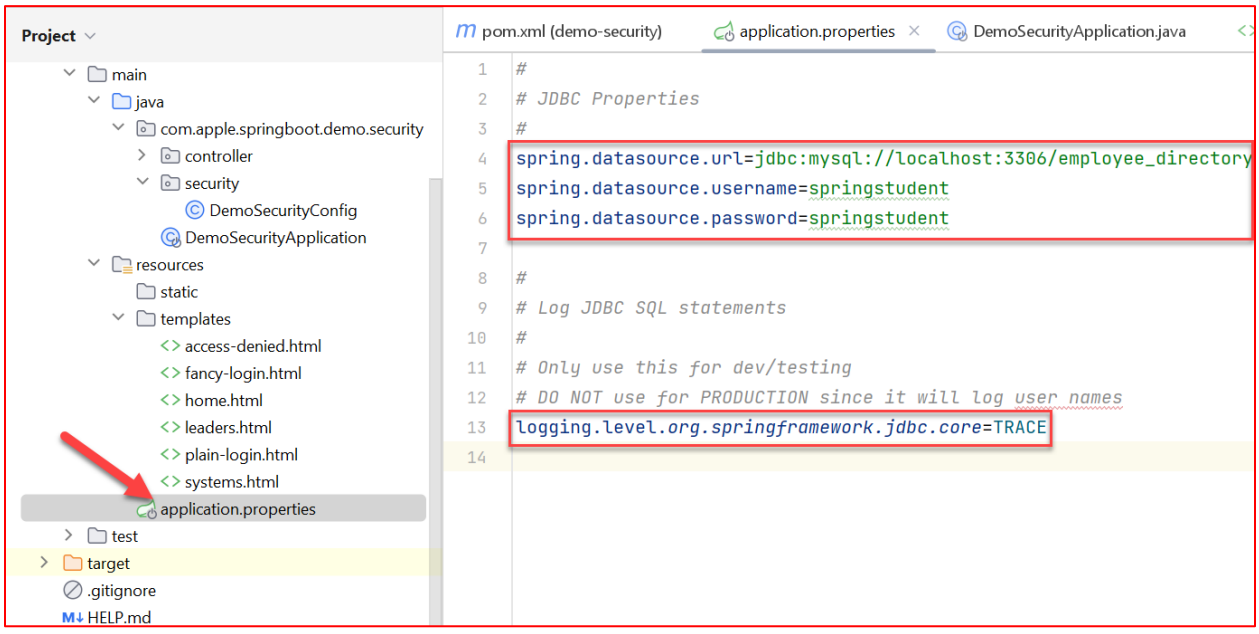
and `'mysql-connector-j'` for the MySQL driver.

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
</dependency>
```

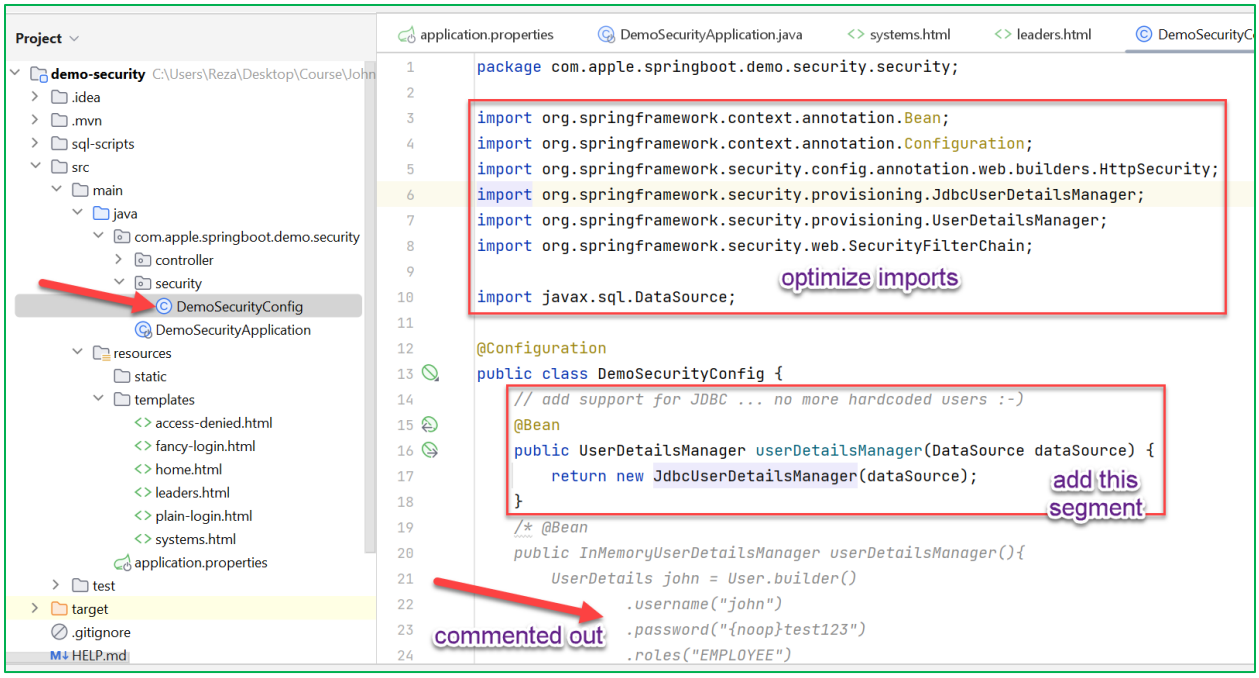
Remember to load the Maven changes for proper integration.



Next, update your `'application.properties'` file with the JDBC connection details. For learning purposes, we'll also enable logging of JDBC SQL statements. This is useful for development and testing but avoid it in production due to security concerns.

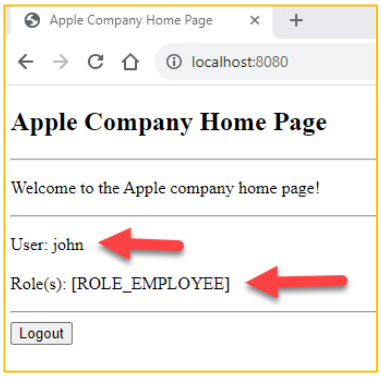


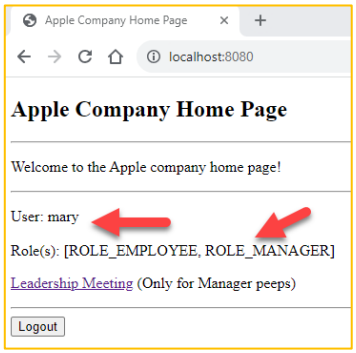
Now, let's modify our Spring Security configuration for JDBC integration. In `'DemoSecurityConfig'`, comment out the hardcoded users section. Instead, we'll integrate JDBC-based authentication. By leveraging `'JDBC UserDetailsManager'`, we ensure that user information is fetched from the database, removing any hardcoding.

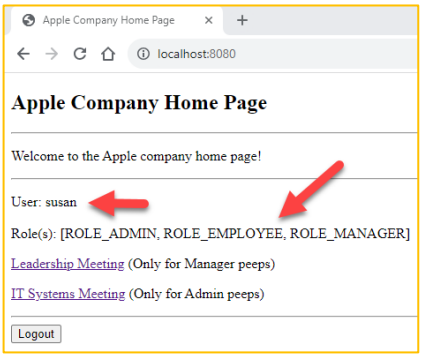


Now, for the moment of truth!

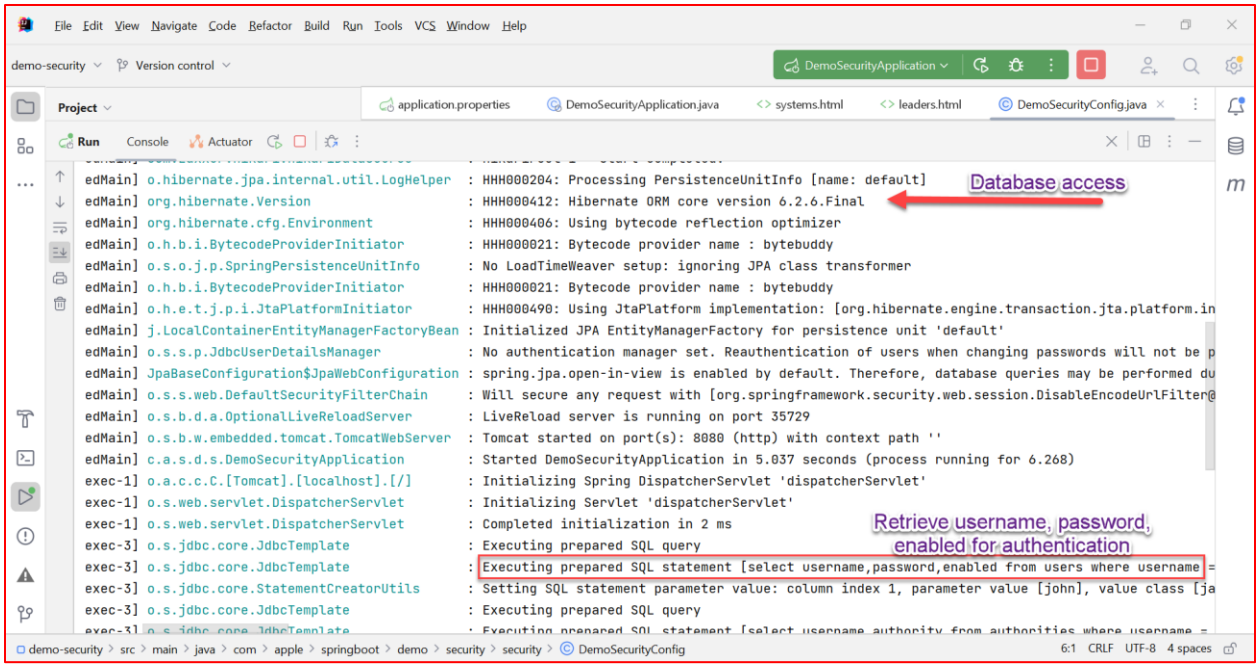
Run your application and navigate to the login page. Log in using one of the user credentials, say `'john'` with the password `'test123'`. If successful, this confirms our JDBC authentication setup.







You can also inspect the logs to observe the SQL queries executed for authentication and authorization.



As a final validation, alter the password for one of the users in the database, for instance, change Mary's password from `'test123'` to `'abc123'`. Try logging in with this new password. If successful, this confirms that our authentication is indeed database-driven.

In conclusion, JDBC authentication offers a dynamic and secure way to manage users in Spring Security, eliminating the need for hardcoded credentials.